

SE

Date
Page No.

NAST

2a) How does Agile Project Management is differ from traditional approaches when scaling to large multi-team environment.

Solⁿ:

criteria	Traditional approach	Agile
Approach	linear, sequential	iterative, adaptive
planning	upfront, comprehensive	continuous, rolling wave
team structure	functional silos, hierarchical	cross-functional, self organizing
Decision making	centralized (project manager)	decentralized (team-oriented)
changes delivery	resistant, managed via board end of project	changes embraced, welcomed incremental, frequent
metrics	plan conformity (budget/time)	value delivery (working software)
communication	documentation, reports	face to face communication -tion, Kanban, scrum

26) How does the prototyping model assist in requirements compared to incremental model.

Soln:

aspect	Prototyping Model	Incremental Model
focus	visualizing and understanding requirements.	Delivering working functional software components.
Validation	occurs via testing a mock or preliminary version (often early).	occurs via testing a functional part of the system (in each iteration)
Nature of output	often throwaway or experimental	always part of the final, delivered product.
Risk focus	reduces ambiguity in requirements.	reduces technical risk through staged iteration.
Flexibility	high: changes are easy in early stages.	moderate: changes can be made in later increments
prototyping: users try it,	answers what the system should do by letting users try it, validating scope & UI/UX.	
incremental:	It validates how the system works by delivering functioning, tested modules, validate functionality	

3b) Compare Behavioral Models and Data Models.

Component	Behavioral Model	Data model
nature	dynamic, process oriented. describes the functional behavior such as how inputs transform into outputs through algorithms. e.g: state diagrams / Use case diagrams	static, structure oriented. focuses on organizing data entities, relationships and data flow in a system, emphasizing the structure and storage.
execution	It executes sequentially.	It follows concurrently.
purpose	best for simulation and design algorithms.	Database schema design and data analysis

4a) How does 'Security By Design' change the architectural approach to software development.

Solⁿ: Security By Design changes the architectural approach of software development by treating security as a core design constraint.

1. Shift from Bolt-on to Baked-in
security controls are embedded directly into the architecture's blueprints, data flows and component interaction models from the very start. This reduces the reliance on perimeter security, focusing instead on internal security and reducing risk of total system compromise.

2. Adoption of shift left philosophy
Security is shifted left moving threat modelling, risk assessment and security requirements to the initial planning and design phases. Architectural flaws are caught before code is written, which is significantly cheaper and more efficient.

3. Implementation of core security principles
principle of least privilege, defense in depth, fail secure defaults, minimizing attack surface.

4. Architecture is designed to handle potential ~~attack~~ anticipated threats

5. Every component communication is treated as untrusted until verified.
6. Continuous security assurance.

4b) Explain the importance of Design patterns in modern software Engineering. How do they contribute to software reuse and maintainability?

Solⁿ: importance;

1. prove solutions

They represent tried-and-tested solutions developed over time by experienced engineers, helping to avoid reinventing the wheel and reducing common pitfalls.

2. Common vocabulary / communication

Design patterns provides a shared language for developers. This improves team collaboration and allows for faster, more concise discussions about architectural design.

3. code quality & reliability

patterns enforce established, clean code structures. Other developers can quickly understand and intent behind the code when recognized patterns are used.

4. Reduced Development Time

Using pre-defined patterns speeds up the design and implementation phase because structure of solution is already known.

Contribution to software reuse and maintainability

1. **Encapsulation of volatile parts**
patterns decouple the parts of code that are likely to change from those that are stable. This allows stable parts to be reused across different applications.
2. **Common Templates**
patterns like factory method or abstract factory provide standardized mechanism for object creation. These factories can be reused.
3. **Component based development**
patterns enable components to be designed as independent, loosely coupled units, making them highly portable & reusable.
4. **Flexibility**
encourages loose coupling. This allows developer to change one part of system without requiring modifications to others.
5. **Simplifying complex systems**
structural patterns provide interface for complex systems, hiding complexity and making maintenance easier.
6. **ease of extension**
patterns like strategy allow adding new functionality at runtime without changing the existing, working code. prevents bugs.

5a) Explain the roles of Alpha, Beta and Acceptance testing in the user testing phase.

Solⁿ:

1. Alpha Testing roles

- conducted by internal staff in a controlled laboratory environment to identify major bugs, functional issues and usability flaws before release to public.
- ensure software is stable and reliable

2. Beta Testing Roles

- conducted by real, potential or actual users in real-world uncontrolled environment. This phase gathers feedback on usability, performance and unexpected usage patterns.
- detect remaining bugs, test the product's performance under real-world conditions and gather user feedback for final improvements.

3. Acceptance Testing roles

- The final phase of Testing, often incorporating or following beta testing, where the client tests the system to ensure it meets predefined business requirements and ready for production.
- formal sign-off that the product is accepted and satisfies business needs.

5b) Why is Reengineering often more cost-effective than developing a new system from scratch?

Ans: Reengineering leverages existing investments and reduces the high risks associated with a total rebuild.

a) Aggravation of Business logic
Legacy system often contain decades of embedded industry expertise and complex rules that are difficult and expensive to discover and document from scratch.

b) Reduced development effort
Reengineering focuses on transforming specific parts of an existing system rather than building every feature and module a new.

c) Lower maintenance cost
Modernizing the code & architecture through reengineering removes technical cost, making system easier and cheaper to support long term.

d) Reduced rework
between 5% - 20% of a new project's total cost is lost to rework. Reengineering starts from a stable, already tested foundation, minimizing these waste efforts.

e) Lower risk of failure
It allows for a safer, staged approach that maintains business continuity.

6a) Difference between Reviews and Inspections in the context of quality management.

Soln:

parameter	Reviews	Inspections
goal/define	evaluate whether a system or process meet requirements by a employees of development team.	review of code by an external to identify bugs and solve it.
goal	assess suitability, progress or technical accuracy	identify and remove defects in a specific work product.
formality	less formal, may or may not use checklists.	highly formal, follows a rigid process with mandatory checklists.
outputs	actions item lists or records of technical discussion.	formal defect lists, summary reports and metric collection
Timing	often held at end of project phases or milestones.	Performed during development to catch errors before they leave a phase

Design Pattern

- Design pattern is communicating objects and classes that are customized to solve a general problem in a particular context.
- Design patterns identifies participating classes and instances, their roles and collaborations and distribution of responsibilities.

Characteristics:

- reusability
- standardization
- efficiency
- flexibility

Types

- a) Creational Design Pattern
- b) Structural " "
- c) Behavioural " "

a) Creational Design Pattern

- it abstracts the instantiation process
 - they help in making a system independent of how its objects are created, composed and represented.
- eg: factory method, abstract factory, singleton pattern, prototype, builder etc.
- used in complex object creation, promotion reuse, reducing coupling

Structural Design Pattern

Advantages

- flexible and adaptable
- centralized control

Disadvantages

- increased complexity
- overhead

b) Structural Design Pattern

- It concerned with how classes and objects are composed to form large structures. They use inheritance to compose interfaces or implementations.

eg: Adapter Method, bridge method, Composite, decorator, facade, flyweight etc

importance:

- making independently developed class libraries work together.
- describes way to compose objects to realize new functionality

c) Behavioral Patterns

- Concerned with algorithms and assignment of responsibility between objects. It describes patterns of class objects and communication between them.

eg: Chain of Responsibility method, command method design pattern, interpreter method, observer method.

importance:

- use inheritance to distribute behavior between classes.
- characterize complex control flow.

b) Adapter: convert the interface of a class into another interface client expects.
Let classes work together

Bridge: Allows the abstraction and implementation to be developed independently.

Composite: characterizes a collection of items that are handled the same way as a single instance of the same type of object.

uses:

- adapting to interfaces
- organizing object relationships
- hierarchical class structures

c) Chain of Responsibility Method
- use to achieve loose coupling in software design where a request from client is passed to a chain of objects to process them.

- Command Method
transforms a request into an independent object with all of information's request.

Interpreter method
defines a grammatical representation for a language and provides an interpreter to deal with grammar.

uses

- communication between objects
- encapsulation of behavior
- interaction between objects

2019, Fall

1 a)

- Considerations during unit testing
- i) Module interface: ensuring that information flows correctly into and out of program unit. It includes checking parameter types, order and number.
 - ii) Local Data structure: examining local data to ensure that integrity is maintained during all steps of execution.
 - iii) Boundary conditions: Testing the module at the limits of its data ranges.
 - iv) Independent paths: ensure that every logical path through the module is executed at least once to uncover logic errors.
 - v) Error handling paths: verifying that the module can gracefully handle incorrect data or unexpected situations without crashing.

Importance of Driver in unit test:

- A main program that accepts test case data, passes it to module being tested and prints result.
- It acts as a controller to trigger the module, allowing developers to test functionality even if the actual calling hierarchy isn't built yet.

Stub:

- A dummy sub-program that replace modules called by the module under test.
- It allow the module to complete its execution by providing minimal simulated responses, preventing the test from failing due to missing dependencies.

16 Control flow diagram

```

int a=1, b=1, n, c;
int i=1;
while (i<=n) {
    c = a+b;
    a = b;
    b = c;
    print ("%d", c);
    i++;
}
    
```

Node1: initialization of variables
 int a=1, b=1, n, c, int i=1

Node2: The loop condition
 while i<=n

Node3: body of loop
 c = a+b, a = b, i++, print c

Node4: exit

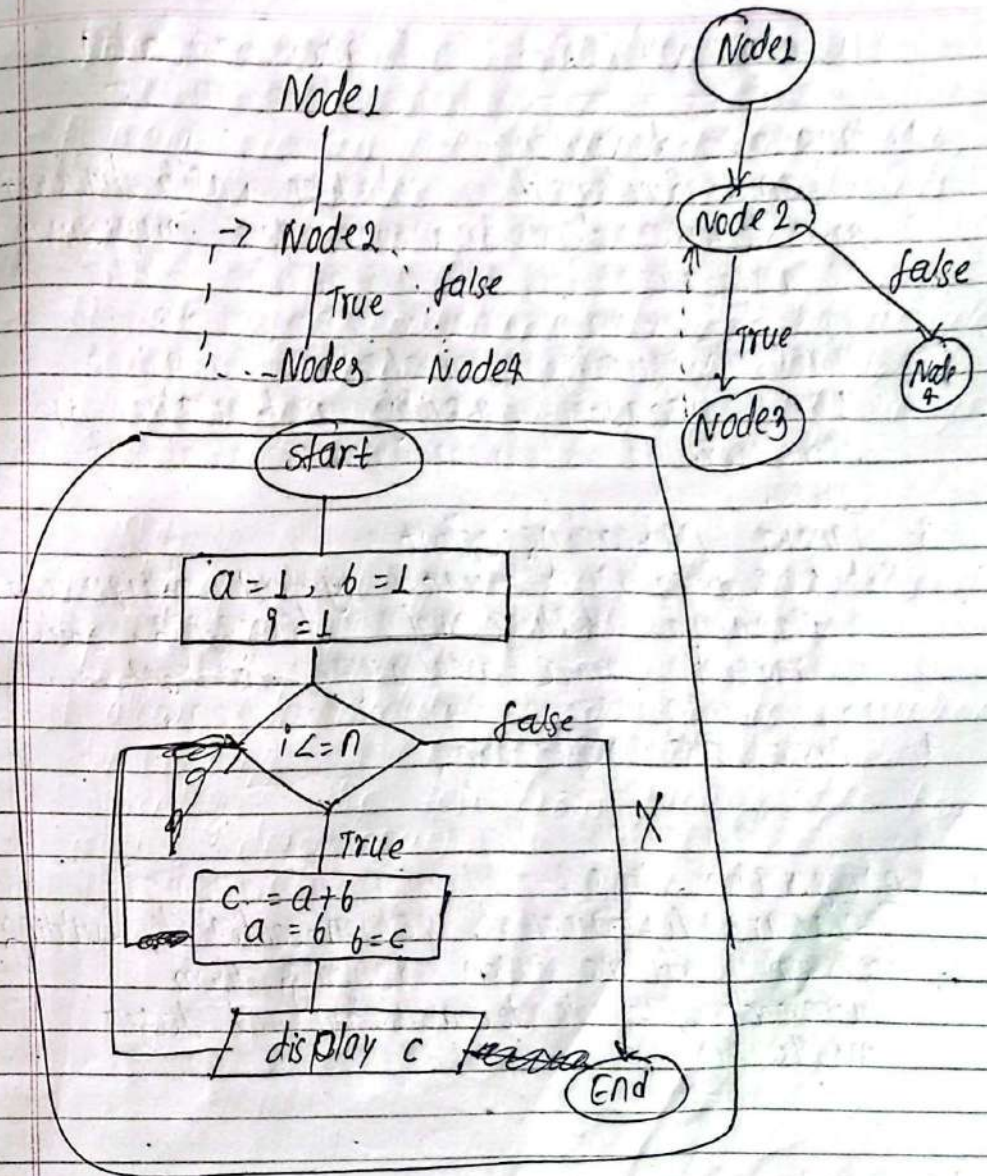
Cyclomatic complexity:

$$V(G) = E - N + 2P$$

$$= 4 - 4 + 2 \times 2 = 2 \quad \#$$

$$V(G) = p + 1 = 1 + 1 = 2$$

E = edge
 N = nodes
 P = 2 Connected
 (Predictor) Components



Elements of LOC (Lines of Code)

1. language dependency
same functionalities takes vastly different no. of lines depending on the programming languages.
2. Lack of quality correlation
LOC measures quantity not quality or efficiency.
3. Ignores non-coding effort
LOC fails to count system design and planning, testing and debugging, code reviews and mentoring, documentation.
4. discourages refactoring

Configuration items:

These are fundamental, version-controlled building blocks of software system ranging from source code file to documentation and design models.

Common problems during Continuous System integration.

1. Data incompatibility
- mismatch data formats or structure between system often caused by connecting legacy systems with modern, cloud-based applications.
unreliable test.
2. inaccurate or outdated tests that fail to detect bugs leading to the deployment of faulty code.
3. Slow performance
ineffective or slow build processes that hinder efficiency, often becoming slower than manual methods.
4. Security vulnerabilities
increased connections between applications can introduce new security risks.

9a) Reasons that the people factors affects the software development process:

→ 1. Skill gaps and inexperience
Software development is a knowledge-intensive task. If team members lack of specific technical skills or domain expertise for project, they will face a steep learning curve. This leads to slow progress, frequent errors.

2. Poor Communication and Coordination
Software is built by team not a individual, if breakdown in communication it leads to misunderstanding, duplication, no collaboration.

3. Motivation and Moral issues.
Low morale caused by poor management, lack of recognition, unhealthy working environment reduces productivity. Issues like human burnout can cause team members to not engage leading to lack, slow.

4. Staff turnover
If personnel leave mid-project. Adding new man-power makes project later. Training new staff is time consuming.

G. Ineffective Leadership and Management
If leadership fails to provide clear vision, resolve conflicts team efficiency drops. Poor estimation by management at start of project can lead to impossible deadlines.

G. Individual Productivity Variance
There is massive gap between the productivity of a star developer and average one. If project was staffed without considering the specific output capabilities of individual involved, original timeline was unrealistic.

SE
ACIT

Ethics for Software Engineers

1. Confidentiality

Engineers must respect the privacy of their clients and employers by not disclosing sensitive data regardless of whether a formal agreement is signed.

2. Competence

Engineers should not misrepresent their level of expertise or take on work that is outside their technical capabilities.

3. Intellectual Property Rights

Engineers must be careful to protect the intellectual property of others and they do not use pirated or unlicensed software.

4. Customer Misuse

Software Engineers should not use their technical skills to misuse other's computers like creating malware or unauthorized access.

5. Public Interest

Engineers should approve software only if it is safe and does not diminish the quality of life or privacy of public.

6. Clear communication

7. Team work

Software Engineers work in team. So working together they must listen their members ideas, take right decision among them and work continuously with collaborations and resolve conflicts.

1b) Cost Estimation Using Empirical Estimation. Empirical model use historical data and mathematical formulas to predict project costs.

They are used by:

1. Sizing the Project: Estimating the size in Lines of Code (LOC) or Function Points (FP).

2. Applying Formulas: Using regression based equation like:
$$Effort = A \times (size)^B$$

3. Adjusting for variables:

Applying Cost Drivers (e.g. developer experience, product complexity) to refine the estimation.

eg: The Cocomo model is a classic empirical approach where projects are categorized (Organic, Semidetached, Embedded) to determine the constant A and B.

Numerical

Risk Identification = 60% success = 40% fail

Risk Probability (P) = 70% = 0.70

Risk impact = 80 components
size = 350 LOC
cost = Rs. 20 per LOC

No. of failed components = $80 \times 40\% = 32$ components

Total cost = $350 \times 20 \times 32 = 224,000$ Rs.

Risk Exposure (RE) = $0.70 \times 224,000 = \text{Rs. } 156,800$ #

2 a) which software model you prefer when software need to deliver within 2 to 3 months. why explain.

Solⁿ: Agile (specially Scrum) or RAD (Rapid Application Development) model prefer.

why:

- a) Iterative Delivery
- b) Flexibility
- c) reduced development time
- d) reusability of features
- e) User involvement

a) Iterative Delivery
These models focus on delivering a 'Minimum Viable Product (MVP)' quickly. You can have a working version of software in weeks not months.

b) Flexibility
Requirements often change that are adjusted by Agile during short lifecycle.

c) High User Involvement
Since timeframe is short, frequent feedback from clients. Face to face communication with clients via scrum. Constant feedback from client prevents wrong development, reducing time wasted on rework.

d) Time Boxing
Scrum uses 2-4 week 'sprints' which fit perfectly to complete project in 2/3 month.

✓ 1. Iterative Nature
Agile breaks the project into small sprints (2-4 weeks). This ensure that a working version of the software is delivered incrementally, fitting perfectly within 3 month window.

2. Focus on Core Features
In short timelines, you can prioritize 'must have' features first, ensuring the client has a functional product

even if some minor features are delayed.

3. High User Involvement
Since time is tight, constant feedback from client prevents 'wrong' development, reducing the time wasted on rework.

4. Adaptability
If requirements change halfway through 2 month, Agile allows the team to pivot quickly.

5a) 3 steps phases of Software quality Review.

1. Preparation

Reviewers study the documents or code individually to identify issues before the formal meeting. Reviewers often use standardized checklist to ensure they don't miss common errors.

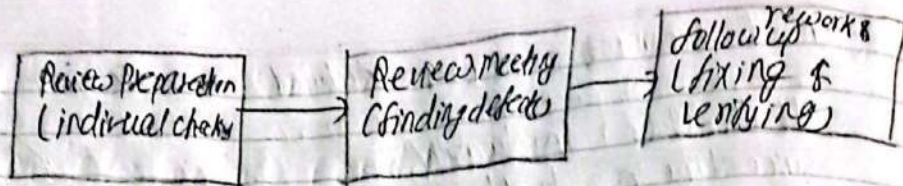
2. Review Meeting (Pre-scan)

The author and reviewers meet to discuss findings, identify defects and suggest improvements.

3. Moderator (Keep track of meeting), recorder (to take notes) & author (to explain)

3. Rework and follow up (Post)

The author corrects identified defects and a coordinator verifies that all issues have been addressed. Once all issues are resolved, review is officially closed. The product is moved to next stage of development.



6a) Sources of change in software project

Units Fundamental Sources of Change

1) New requirements: stakeholders request new features, clients needs new functions.

2) Environmental changes: OS updates, new hardware

3) Bug Fixes: Error found after deployment

4a) 156

q. a) Write down steps for basis path. Calculate cyclomatic complexity.

```

while (condition) {
  if (condition) {
    if body
  }
  else if (condition) {
    else if body
  }
  value update
}
  
```

Soln:

Steps for basis path:

1. Construct the Control flow diagram
Convert the code logic into nodes (statement) and edges (flow).

2. Calculate Cyclomatic Complexity
This identifies the no. of independent paths through the code.

3. Identify the Basis set
Determine the specific paths that must be executed to ensure every statement is covered at least once.

4. Design Test cases
Create input data that forces the program to follow each identified path.

3. path1: Node 1 → Node 2 (loop condition false)
path2: 1 → 2 → 3 → 6 → 1 (if true)
path3: 1 → 4 → 5 → 6 → 1 (elseif true / if false)
path4: 1 → 2 → 4 → 6 → 1 (both if/else false)

Calculation for code snippet:

Node 1: while condition

Node 2: if condition

Node 3: if body

Node 4: else if condition

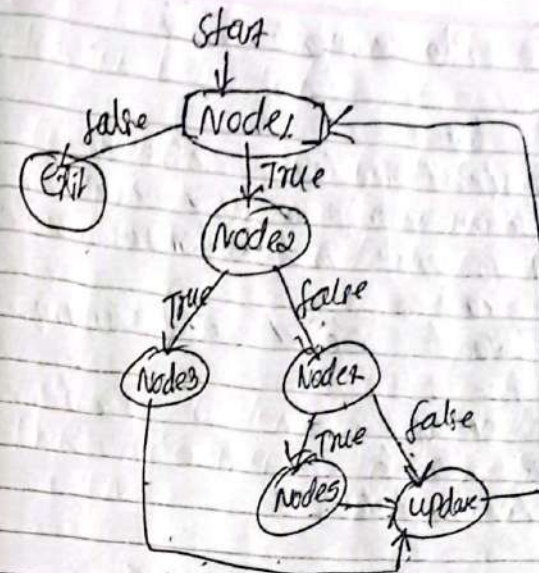
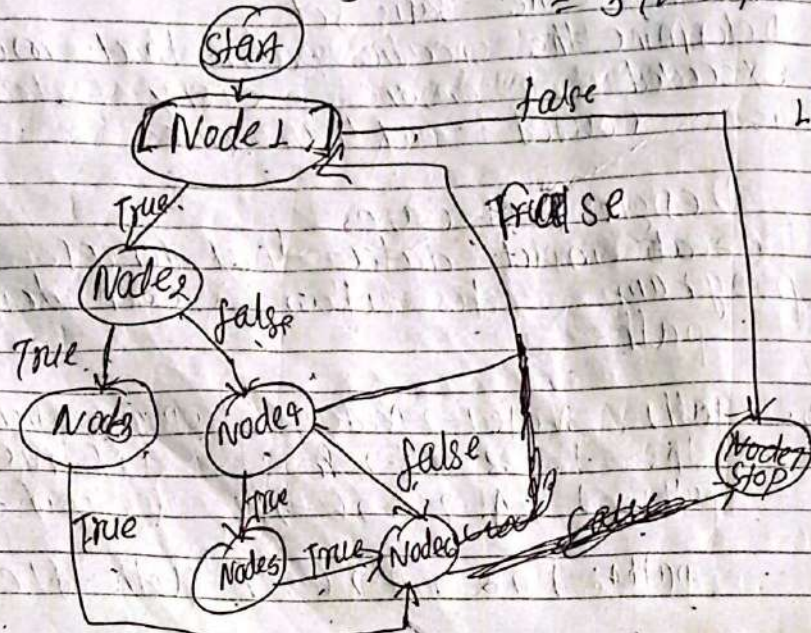
Node 5: else if body

Node 6: update

Node 7: stop

(i) Predict node

Cyclomatic complexity = $V(G) = P + 1$
 $= 3 + 1 = 4$



(ii) Graph Theory

Cyclomatic complexity = $E - N + 2P$
 $= 9 - 7 + 2 \times 1$
 $= 4$

under which

c. Region method

Regions = 3 enclosed loop + 1 outer space
 $= 3 + 1 = 4$

39

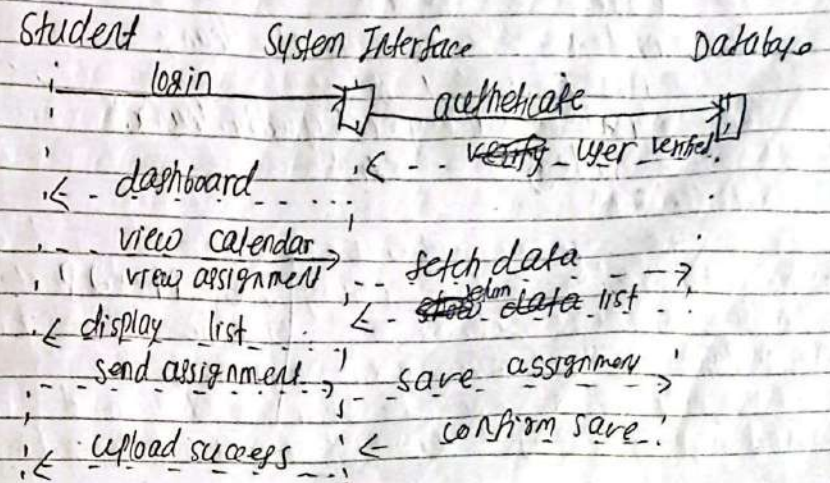
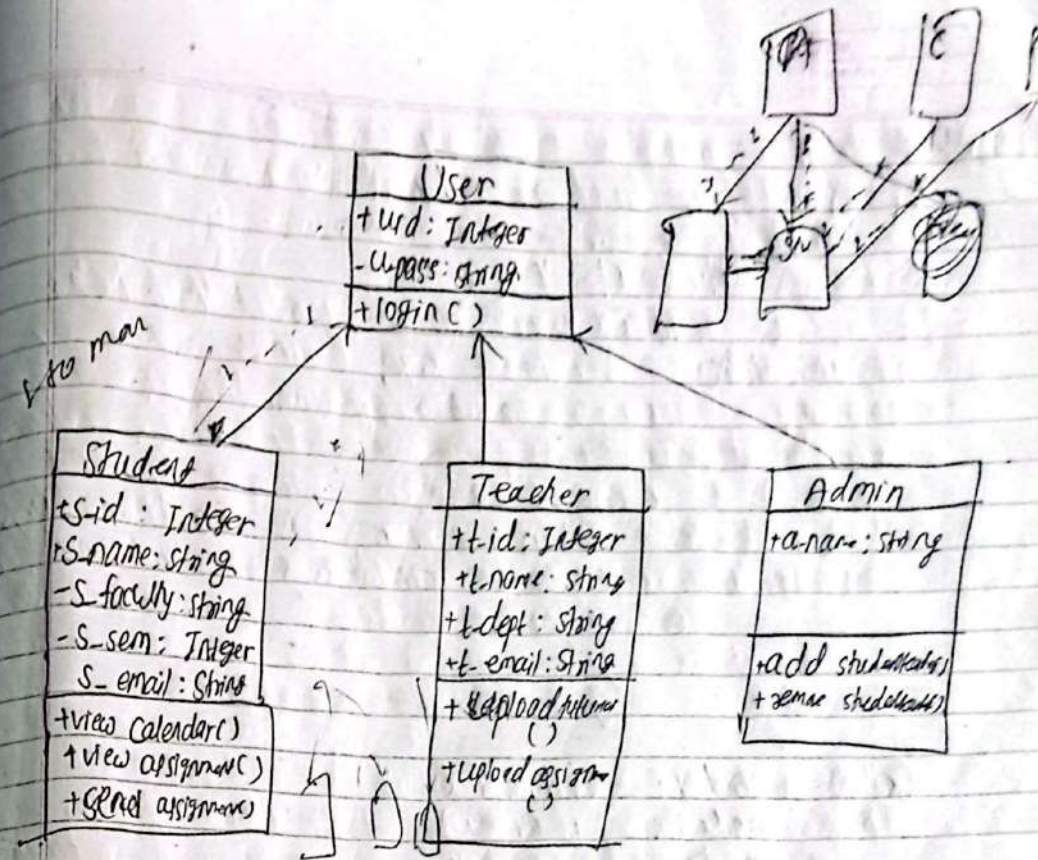
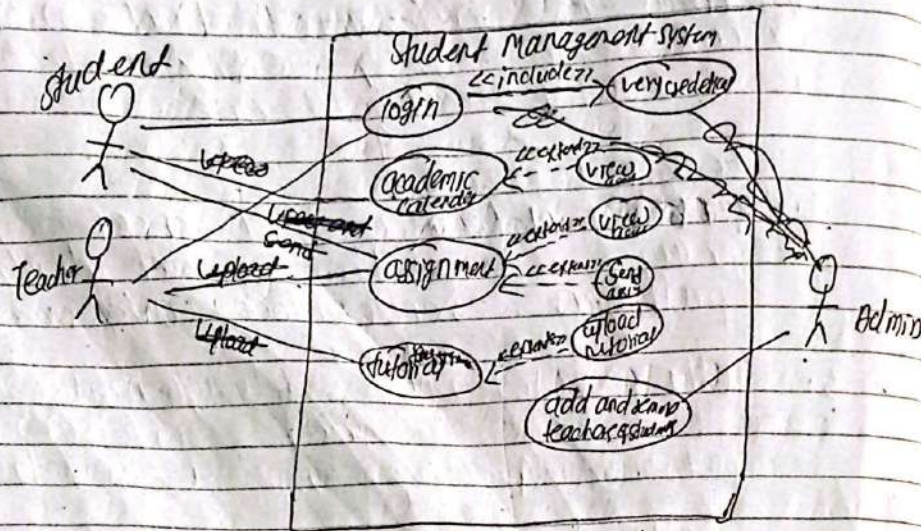
Student Management System

- Students to login
- " to view academic calendars
- " to view and send assignments
- college admin to upload add, remove students & teachers
- Teachers to upload tutorial and assignments

System also stores

- Student details
- Teacher details

1) Draw use case, class, sequence.



cos Nos.

4. int fun (int x, int y)

while (x != y) {

if (x > y) {

x = x - y;

print "x is greater than y";

}

else {

y = y - x;

print "y is greater than x";

}

}

}

Soln: node1: while (x != y)

node2: if (x > y)

node3: x = x - y

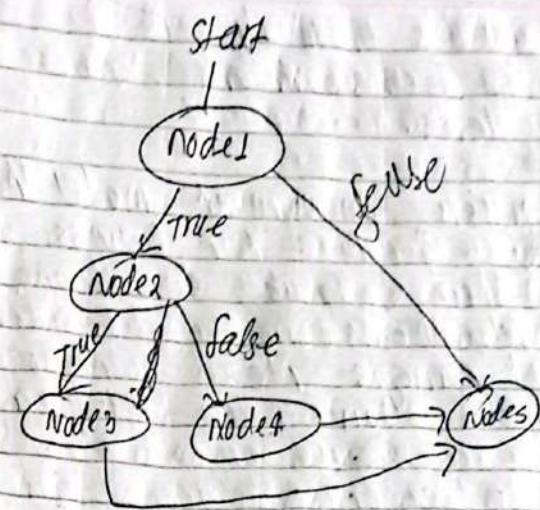
node4: print "x is greater than y"

node5: y = y - x

node6: print "y is greater than x"

node7: exit

$$E - N + 2P$$



$$6 - 5 + 2 \times 1$$

3

Cyclomatic complexity:

P = node1, 2.

(i) using predicate node

$$V(G) = P + 1 = 2 + 1 = 3$$

(ii) graph theory

edges, no. of nodes, prediction

$$V(G) = E - N + 2P = 6 - 5 + 2 \times 1 = 3$$

on closed loop + out of space

$$(ii) \text{ region theory} = 2 + 1 = 3$$

2. Explain Capability Maturity Model Integration (CMMI) framework and describe its five Maturity levels.

→ It is a process improvement model that helps organizations to achieve their business goals by improving their software engineering capabilities. It is developed by Software Engineering Institute (SEI) at Carnegie Mellon University and it is based on the best practices of successful software organizations.

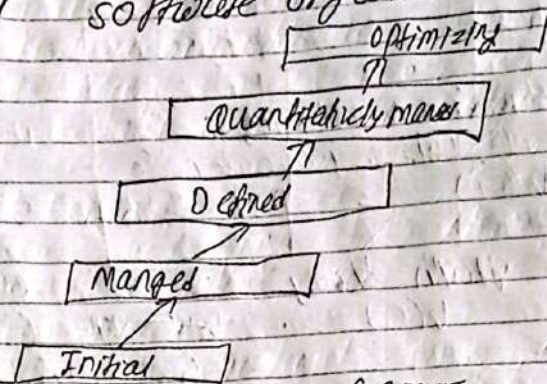


Fig: Level of CMMI

Level: Initial

- Process are unpredictable, poorly controlled and ineffective.
- The organization relies heavily on individual heroes, and success depends on specific, often heroic, efforts rather than proven processes.
- Projects often exceed their budget and schedule and past successes are difficult to repeat.

Level: Managed

- Basic project management practices are established.
- Processes are planned, documented, performed, monitored and controlled at project level.
- Projects are consistent and can be repeated.

Level: Defined

- Processes are standardized across the organization.
- Processes are proactive, providing a consistent framework for all projects.

Level: Quantitatively Managed

- Processes are measured and controlled using statistical and other quantitative techniques.
- It is data driven with specific performance metrics to understand and predict process performance.
- High quality and low risks, as process variations are analyzed and corrected.

Level: Optimizing

- Continuous process improvement is practiced.
- Processes are constantly improved based on quantitative feedback and innovation.
- The organization is stable, flexible and able to respond to change rapidly.

2. Explain Interface Design, Component Design and Database Design in detail. How do they contribute to the development of high quality software systems?

Solⁿ: 1. Interface Design
Interface Design involves how the software communicates with itself, with other systems and users.

User Interface (UI): focus on usability, look and feel.

External Interfaces: Defines how a software interacts with other hardware or software systems (e.g. APIs).

Internal Interface: Manages how data flows between different modules within the software.

2. Component Design

This involves identifying the internal logic, data structure and algorithms for individual software modules (components).

- It transforms the structural design into a blueprint for coding.
- It follows principles like modularity, independence and high cohesion.

3. Database Design

It involves creating a data model that supports the software's business logic.

- It includes defining schema, tables, relationships, storage structures.
- This includes data integrity, security, minimizing redundancy through normalization.

Contribution to High quality software

1. Maintainability:
Well designed components are easier to fix or update without breaking the whole system.

2. User Satisfaction
Good interface design ensures the product is usable and meets user expectations.

3. Scalability:
Solid databases and component structures allow the system to handle more data and users over time.

4. Reliability
Proper design reduces the errors and data corruption.

#3. How change control helps in maintaining the integrity through its lifecycle:-

→ Version Control: It prevents 'overlapping' where one developer accidentally overwrites another's work.

→ Traceability: It provides a clear history of who changed what and why, which is vital for debugging and audits.

→ Stability
By filtering out unnecessary or risky changes, it ensures the software remains stable and functional throughout its lifecycle.

4. Explain the importance of validation testing in software development.

Soln:

1) Meets User Needs
It ensures the final product actually meets user expectations and requirements.

2) Risk Mitigation
Identifying gaps between requirements and actual product early that prevents costly rework after release.

3) Compliance
For many industries (like medical or financial) validation is legal requirement to prove the software is safe and effective.

4) Customer Confidence
Successful validation serves as a 'final check-mark', giving stakeholders confidence that the software is ready for production.

seiter
4. Why is Alpha testing referred to as 'Developer-site Testing' and Beta testing as 'Field Testing' with suitable example.

Soln: Alpha testing is referred to as 'Developer-site Testing' because it is performed by testers who are usually employees of organization. It takes place in a controlled environment at the software company's own office. Goal is to catch major bugs in a safe environment where developers immediately observe and fix them.

example: A game studio develops a new RPG. Before showing it to public, the internal QA team plays game in the studio to find 'game-breaking' bugs or crashes.

Beta Testing is referred as Field Testing as it occurs in the real world environment (field) where software will actually be used. It is performed by clients who aren't part of the organization. It uncovers issues that might not appear in a lab setting. It concentrates quality of product and ensures that product is ready for real time user.

example: After internal team fixes major bugs, the game studio released a "Closed Beta" to 5,000 players globally. The players test the game on their own PCs to see if the server lag.

or if the game crashes on specific graphics cards.

Exercise Differentiate between OOA and OOD.

features	Object Oriented Analysis	Object Oriented Design
Primary goal	To understand what the system does.	To define how the system will be built.
Definition	Engineering approach that focus on identifying and modeling objects within a system and their interacting	This is implementation of the conceptual model developed in OOA like defining data structure, algorithm & software architecture.
Output	produces a requirement model e.g. Use case diagram, class diagram showing real world objects.	Produces a technical blueprint (e.g. detailed class diagrams with attributes, methods, sequence diagrams).
Abstraction	High level abstraction, ignores technical constraints like programming language or database.	Low level abstraction considers constraints like memory, performance and specific technologies.

5a)

Node1: while (x < y)

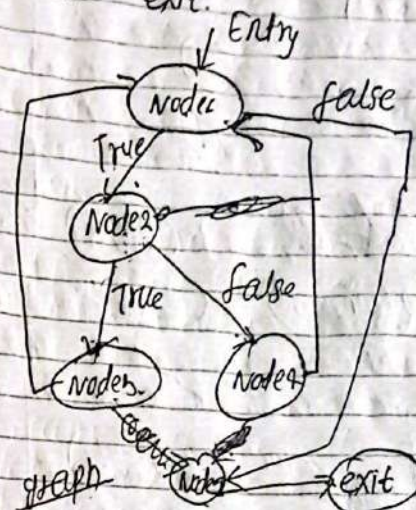
Node2: if (x == 3)

Node3: printf(" ");

Node4: printf(" ");

Node5: x++

Node6: exit.



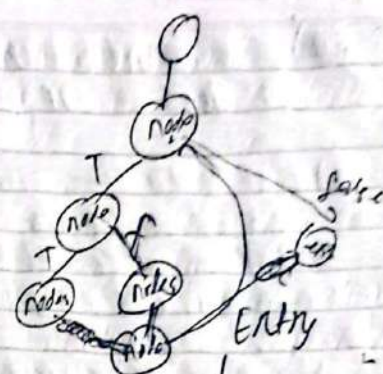
$$V(G) = E - N + 2P$$

$$= 6 - 5 + 2 \times 1 = 3$$

(i) region

$$= 2 + 1$$

$$= 3$$

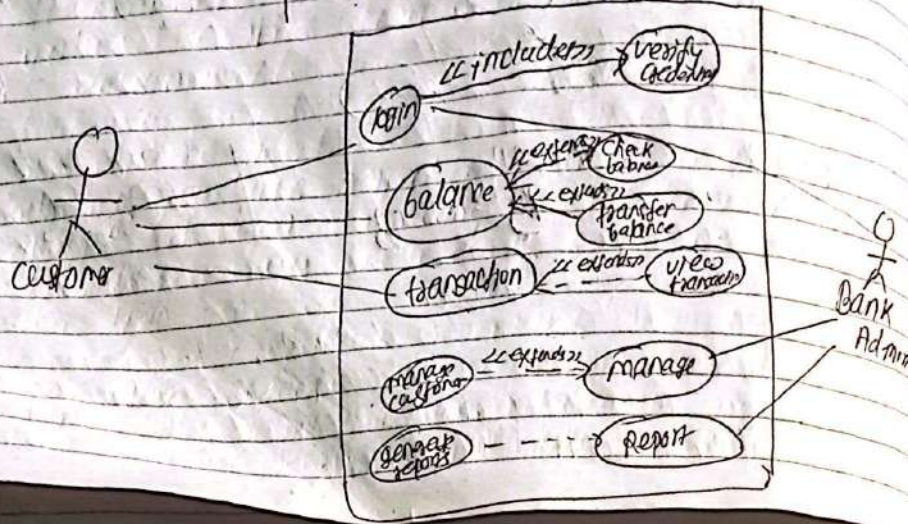
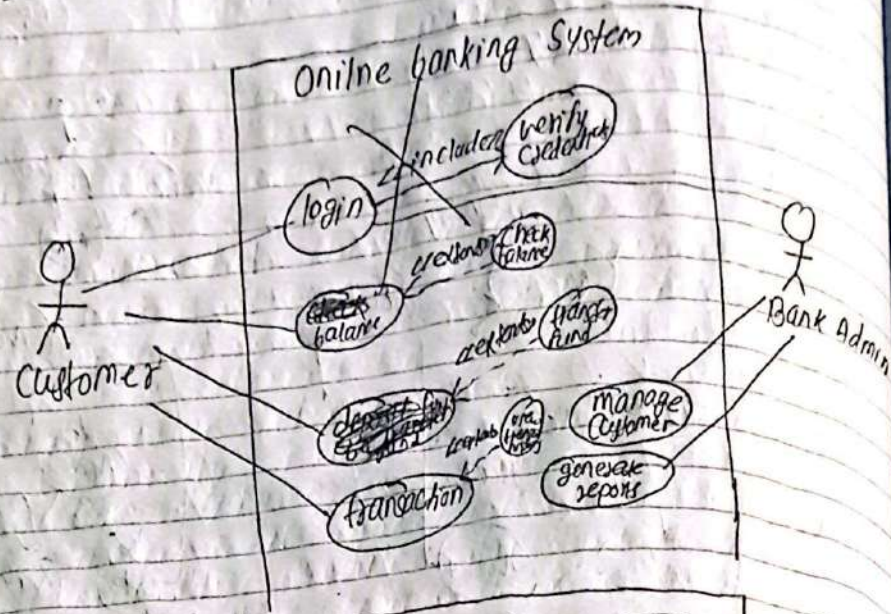


Using predicate

$$V(G) = P + 1$$

$$= 2 + 1 = 3$$

3a) Draw use case and DFD level 0 and level 1 for online banking system.



DFD 0

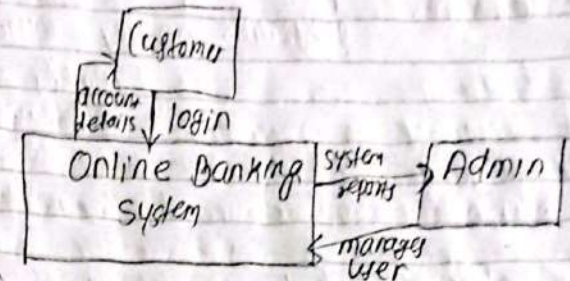


fig: DFD 0 for Online banking system

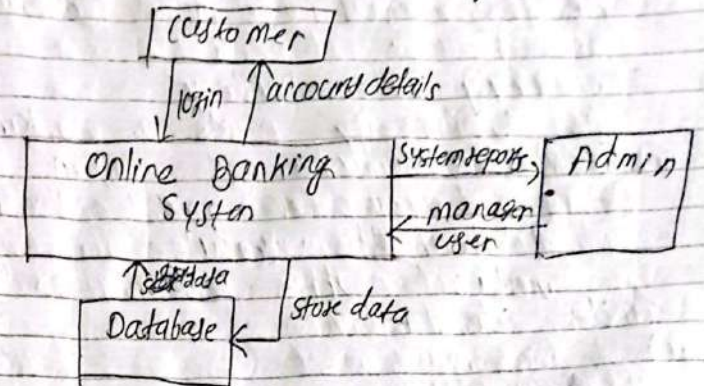


fig: DFD 1 for online banking system

2a)

Pokhara University software application
programs notices extracurricular information
6-9 months
sharing only

other - 2 years
purpose software development model justify.

Soln: Incremental Development Model
(executed using an Agile framework like Scrum) is most suitable choice.

1. Early value Delivery

The University needs a working module for 'Information sharing' within 6-9 months. This Incremental model allows the team to deliver this module first, providing immediate value while the rest of the system is still under development.

2. Risk Management

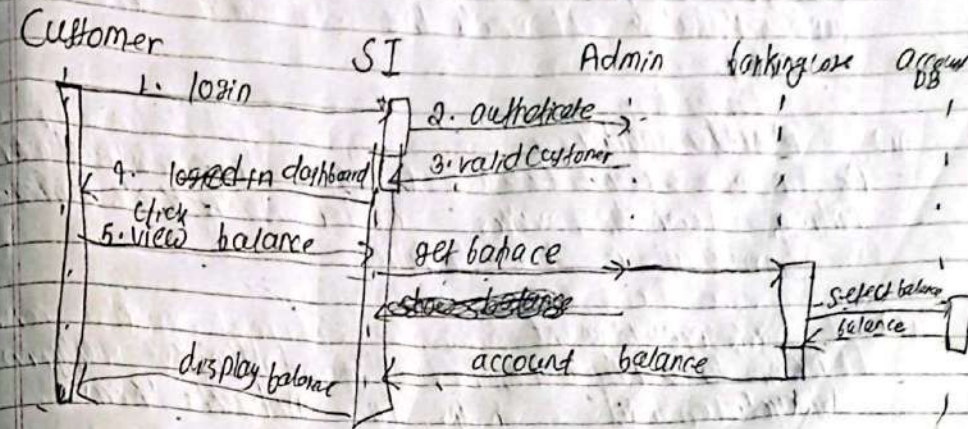
Developing a large project 2 years is risky. By breaking it down, team can identify technical or requirements issues early during 6-9 month phase rather than end of 2 years.

3. Accommodating Change

University requirements can be change in timely. Since the rest of the features will take 2 years, an

incremental/agile model allows university to refine the requirements based on feedback from 'Information sharing' launch.

4. Parallel progress
While the first increment is used by student and staff, the development team can work on subsequent increments (e.g. enrollment, grading or extracurricular modules) without disrupting the live environment.



NEC

1a) We need to follow software process during software development.

i) Predictability

It provides a roadmap for software development. Making schedules and budgets more reliable. We can calculate resources to expense what the user wants.

ii) Quality Assurance

Before launching a product testing is done which debugs defects and make software error-free and validated. It meets the standards and maintains quality.

iii) Maintainability

As proper documentation is done. There is structured coding which makes easier to update software product in the future.

iv) Risk management

It helps to identify risks, problems, bottlenecks early in lifecycle and helps to mitigate them.

Agile better for frequent change.

- instead of trying to define everything upfront, they deliver software in small increments. Allows customer to provide feedback and change requirements at end of every sprint or iteration without disrupting the

(like waterfall)

entire project.

1b) Benefits of Cost and Time Estimation during project planning.

Solⁿ:

i) Resource Planning

Helps to decide how much resources will be used. How many developers, what tools required.

ii) Budgeting

It provides a baseline for project fund. How much money, economic resources required.

iii) Setting Expectations

Helps manage the client's expectations regarding delivery date.

2a) Use case helps to understand the system requirements.

- Use case describes the interaction between Actors (users) and the system.

- It tells the boundary by identifying Actors (people or other system) outside the system. It identifies stakeholders by asking "who uses this system?" and "who provides information to it?".

- Use cases help to identify error scenarios and

2b) What is SRS (Software Requirement Specification) document. Explain components of SRS document.

Solⁿ: SRS is a document that describes the nature of software system. SRS is a formal, structured document that describes what a software system should do and how it is expected to perform. It describes the nature of software to be developed, acting as a contract between the client and developers.

Component of SRS:

1. Introduction
It explains the purpose, scope and definition of software and its intended audience. It ensures all stakeholders understand technical terms.
2. Overall Description
It describes intended audience's technical expertise and their roles. Product perspective, function and user characteristics.
3. Functional Requirements
It describes the detail input, process and output. It details the system actions such as user authentication, data processing and report generations.

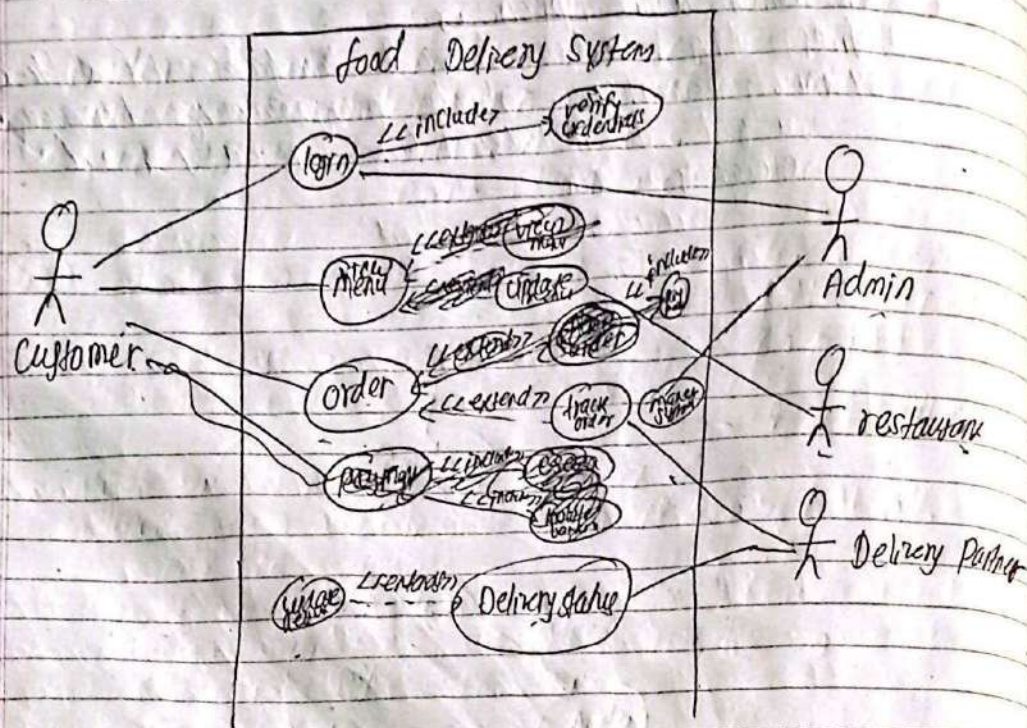
4. Non functional Requirements
It describes the performance, security and reliability of software. It specifies response time, access control, fault tolerance.

5. External Interface Requirements
Details how software interacts with other system and user. User, hardware or software interface.

6. Constraints and Appendices
Defines the boundaries and limitations of software system. Describes which programming language, budget. It includes DFD, use cases and references.

alternative scenarios. This ensures robust requirements engineering

1. defining system scope and boundaries
2. Capturing functional Requirements
3. identifying exception and alternative flows
4. facilitating stakeholder communication



3a) Explain how Coupling and Cohesion helps in effective modular design.

Soln: Cohesion: It measures how closely elements are closed within a single module are related to one another.

- High cohesion ensures that a module performs a single, well-defined task.
- It makes module easier to understand, test and maintain.

Coupling: It measures the degree of interdependence between different modules.

- Low coupling ensures that modules are independent and know as little as possible about each other's internal workings.

It prevents a ripple factor effect. Makes system more flexible and easier to debugging.

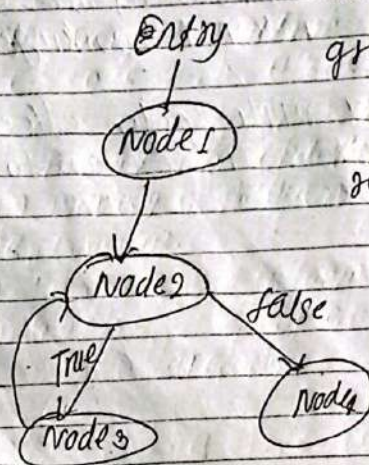
4a)

```
int a = 1, b = 1, n, c;
int i = 1;
while (i < n) {
    c = a + b;
    a = b;
    b = c;
    printf("%d", c);
    i++;
}
```

Node 1:

```
int a = 1, b = 1, n, c;
Node 2: while (i < n)
Node 3: c = a + b; a = b; b = c;
        printf("%d", c);
        i++
```

Node 4: exit



$$VCO = P + L = L + 1 = 2$$

$$\text{graph} = E + N + P$$

$$= 4 - 4 + 2 \times 1 = 2$$

$$\text{region} = L + L = 2$$

4b) What is black box testing? Explain Equivalence Partitioning with an example?

Soln: This (Black box) testing is a software testing method where the internal structure, design, implementation or function are tested of items being tested are not known to tester. It is testing of functions or procedure. The testing focusing on inputs or outputs.

Equivalence Partitioning

It is a black box software technique. EP is a technique to divide a input data of a software unit into partitions of equivalence data from which test cases are derived. The idea is that if one value in a partition passes, all other in that partition will also pass. This test help testers achieve comprehensive test coverage while avoiding redundant tests.

Example: A field accepts an age between 18 and 60.

partition 1: Age < 18 (e.g. 10)

partition 2: Age 18-60 (e.g. 20)

partition 3: Age > 60 (e.g. 70)

5a) Formal Technical Review (FTR)
- A software quality control activity performed by software engineers to improve quality of product.

Objective of FTR

- To find errors in function, logic or implementation.
- to verify that software meets requirements.
- to ensure software is represented according to predefined standards.
- to achieve software that is developed in a uniform manner.

6b) Version Control process supports in SCM.

→ SCM uses version control to track different iterations of code. It allows developer to:

- maintain history of changes
- rollback to previous stable versions if new change break the system.
- supports parallel development (branching) where multiple people work on different features simultaneously.

Unit 1

1b) Advanced AI-Powered healthcare system

Solⁿ: The spiral model is preferred as it is risk-driven process model that combines the iterative nature of prototyping with controlled and systematic aspects of linear sequential model. It is specially designed for large scale, complex and high stake projects.

Spiral model is best because:

i) Risk management

The prompt highlights significant risks and multiple uncertainties. It includes a dedicated Risk Analysis phase in every iteration. This allows the team to identify and mitigate technical or clinical risks early before they become catastrophic.

ii) Handling complexity and scale

With a 3-years timeline and team of 50 developers, data scientists and domain experts, the project needs a structured & flexible framework. Spiral model provides a platform to keep a large team aligned over a long duration.

iii) Iterative Development & Refinement
Spiral model is inherently iterative, each cycle through spiral produces a more refined prototype or version of system that allows evolving requirements common in AI development.

iv) Integration of Domain Expertise
In healthcare domain experts (doctor, clinicians) are vital. Spiral model allows for constant feedback loops where experts can evaluate prototypes at end of each iteration, ensuring AI system meets clinical standards.

v) Flexibility for AI Development
AI projects often involve experimental data science phases where outcome is uncertain. This spiral model's flexibility allows the team to change direction if a particular AI approach isn't yielding the required accuracy or safety.

3b) Class Diagram static structural diagram that is class diagram is a part of UML (Unified Modelling Language). It describes the structure of a system by showing its classes, their attributes, methods and relationships among objects.

Components:

Class: Represent entities or concept (e.g. user, product)

Attributes: data or property of a class (e.g. name, price)

Methods: The actions or behaviors of a class can perform (e.g. login, addCart());

Relationships: connections like association, aggregation and inheritance.

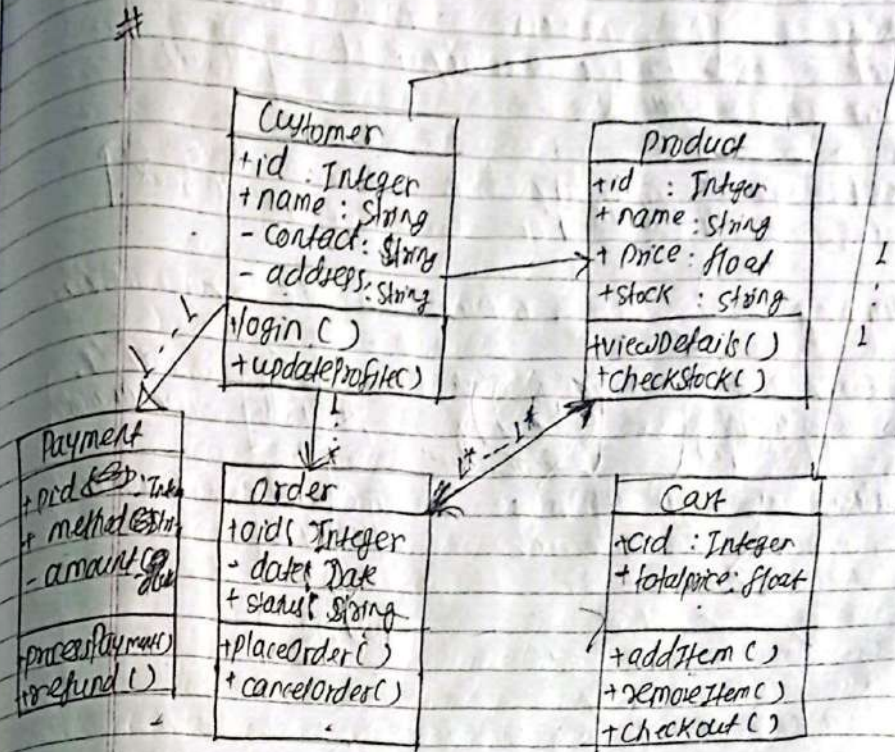
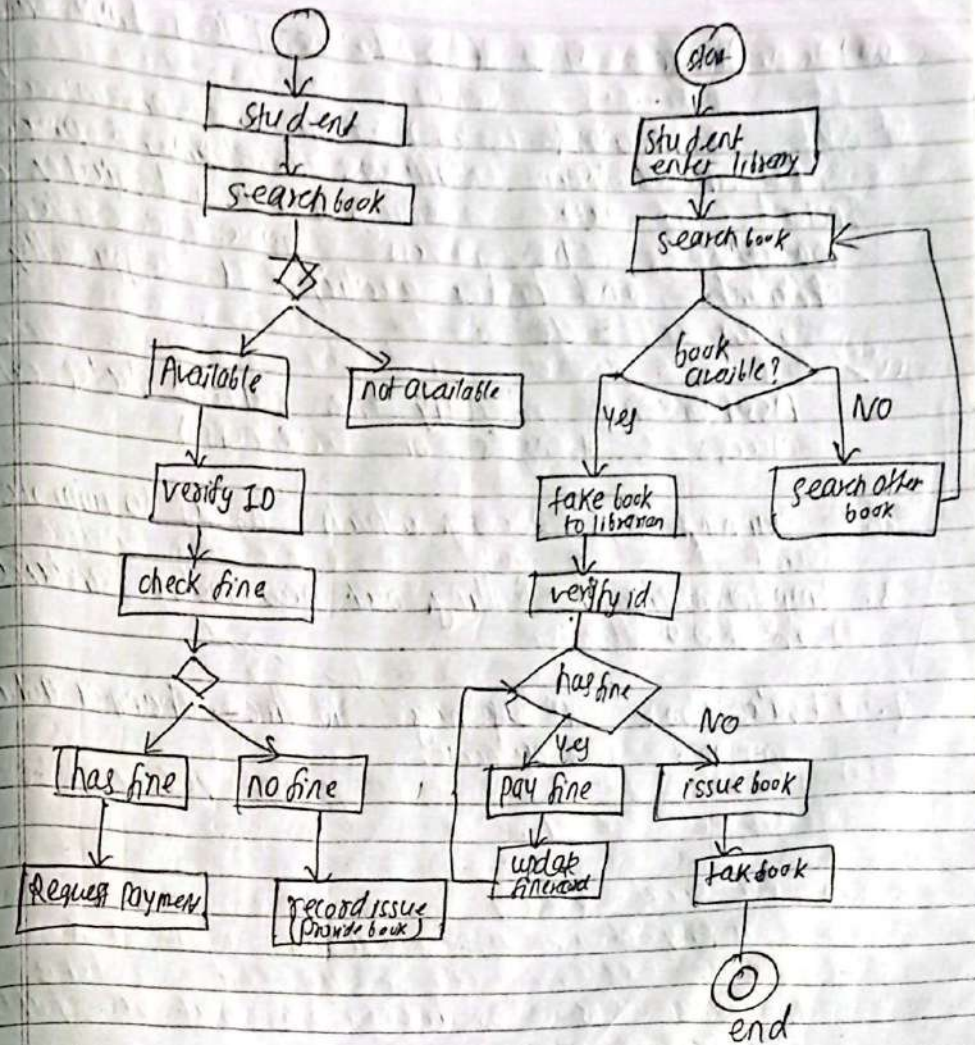
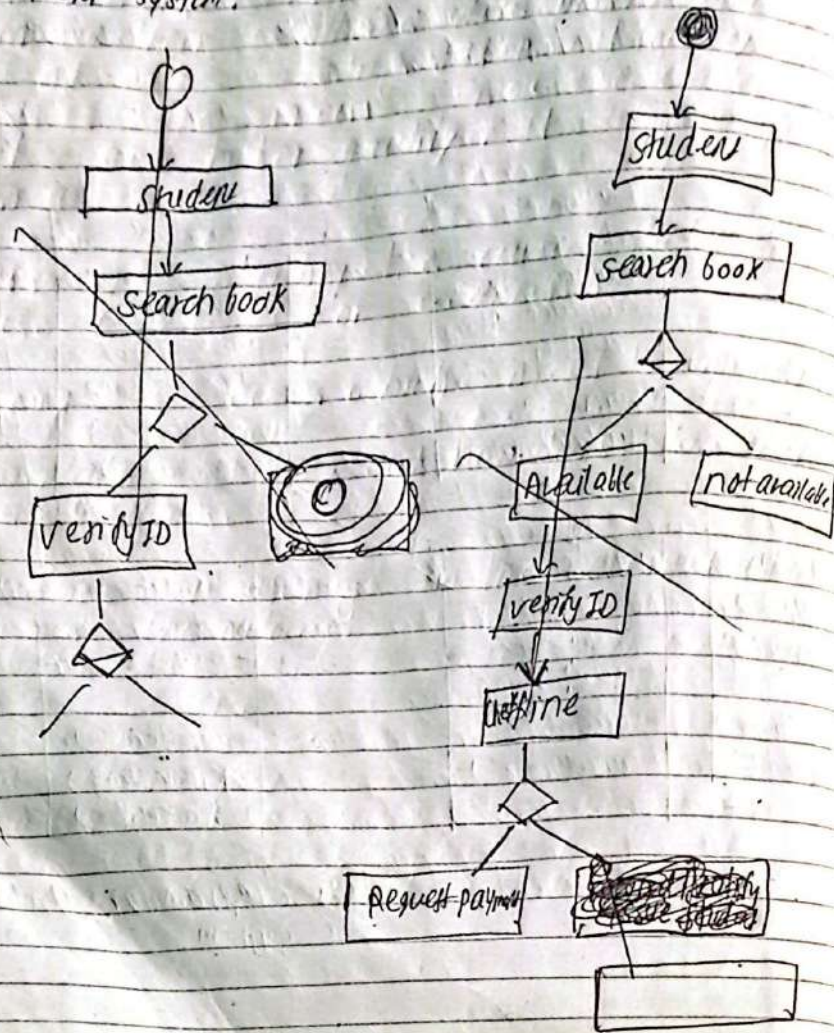


Fig: class diagram for Online shopping system

Activity Diagram to borrow books from your respected college library management system:



6a) Discuss SPI (Software Process Improvement)

SPI framework is a roadmap for organization to improve their software development processes. Instead of just coding faster, it focuses on making the process more predictable, efficient and higher quality. It typically follow a cycle:

Universal

1. Assessment: identify the current state of process (strengths or weakness)
2. Planning: Deciding which areas to improve first.
3. Implementation: Rolling out new tools, standards, or workflows.
4. Evaluation: Measuring if the changes actually improved the outcome.

6b) How Version Management keeps track of all the different versions of software components (like files, codes or documents)

Sol: tracks:

- a) The Central Repository
All files and their complete history are stored in a database called repository. Instead of just saving the every change ever made.
- b) Unique version Identifiers
Every time a change is committed, the system assigns a unique identifier. This could be version number (e.g. v.1.01), a timestamp.
- c) Baselines
A baseline is a snapshot of all components at a specific point in time (e.g. Release 1.0). This allows developers to recreate the exact state of entire project as it existed on a specific date.
- d) Branching and Merging
Branching allows developer to divert from main line of code to work on a specific feature or fix without affecting others.
Merging combines changes back into the main codebase while resolving any conflicts between different versions.

e) Concurrency Control
It manages scenarios where two people edit same file simultaneously.
Most modern system use a 'Copy-Modify-Merge' model, where the system identifies overlapping changes and asks the user to merge them, ensuring no work is overwritten.
It manages tells who made changes, what specific lines were added or deleted, why the change was made.

Universal

Spiral Model is good for risk-driven environment.
as:

i) Risk assessment

ii) Iterative Development & Refinement

iii) Flexibility: It allows change in requirements as project evolves.

iv) Financial Safety

If a project is more risks during risk assessment phase, it can be cancelled before massive amounts of money are spent on full-scale development.

Universal

2a)

Reasons for Agile Popularity

i) Adaptability: Agile allows teams to respond to changing requirements, even late in development cycle.

ii) Customer Satisfaction
Frequent delivery of working software and constant feedback ensures the product meets user needs.

iii) Faster Time to Market
By breaking project into sprints usable features are released much sooner than in traditional models.

iv) High quality
Continuous testing and integration help debug early.

v) Transparency
Regular meetings keep all stakeholders informed and aligned.

26) Online Learning System (OLS)

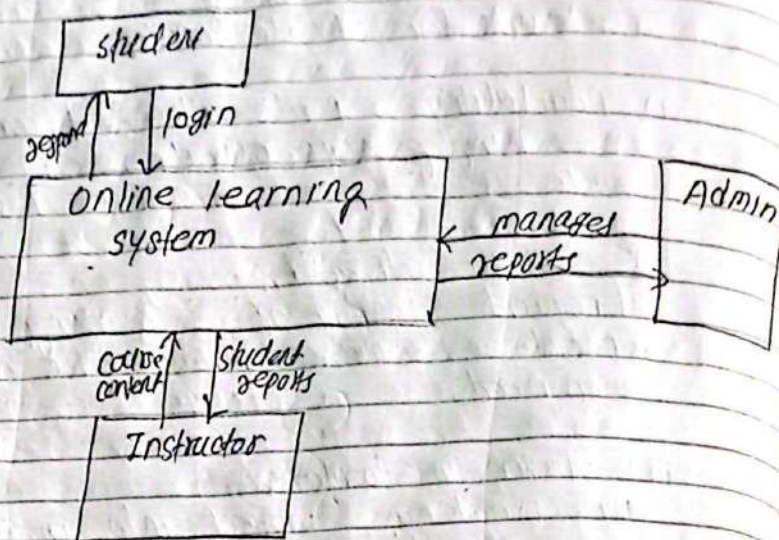
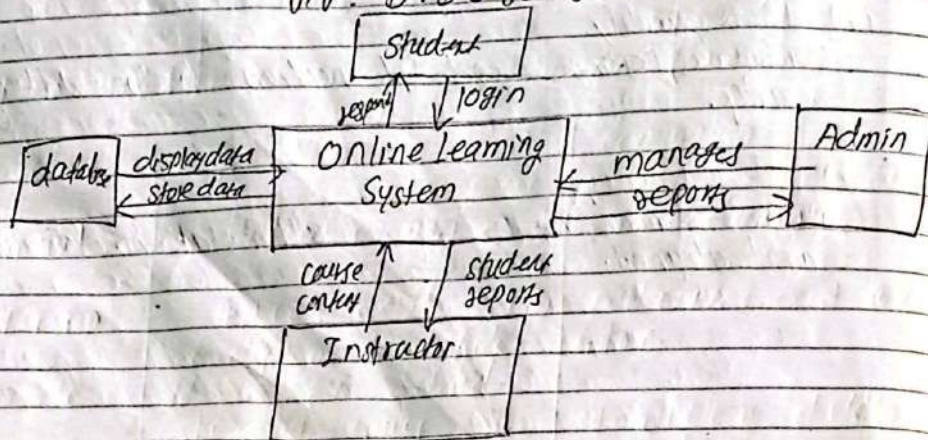


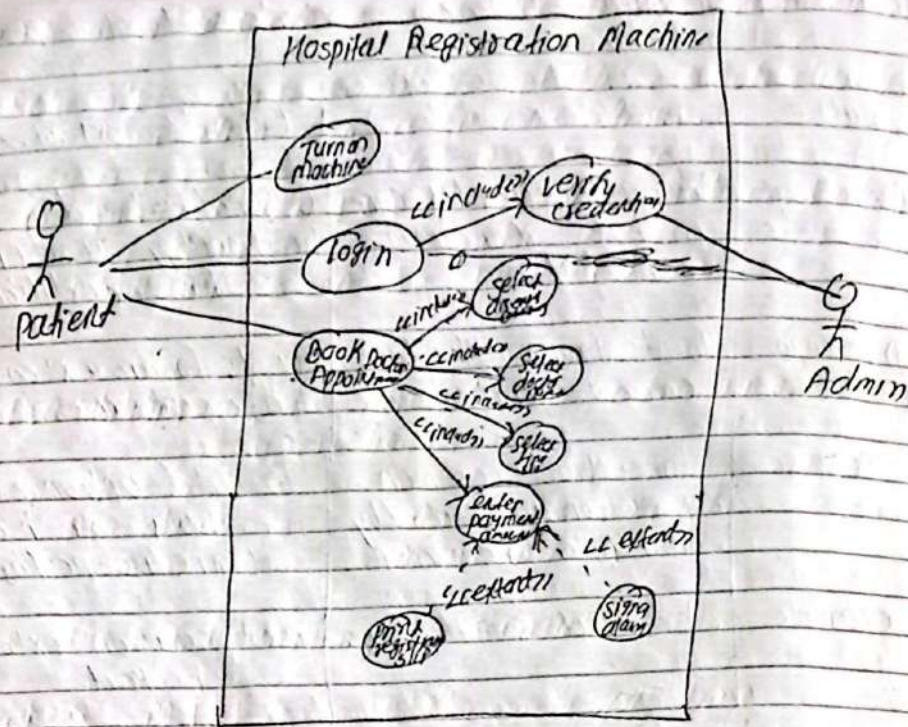
fig: DFD for OLS



3. a) Differentiate between include and extends relations.

Qn:

feature	Include (<<include>>)	Extend (<<extend>>)
Necessity	Mandatory. The base case cannot function without it.	Optional, it adds functionality under specific conditions.
Direction	Points from the base case to included one.	Points from extension to base use case.
Purpose	to extract common functionality used by multiple use cases.	To add optional features or handle alternative paths.
example	checkout includes login	apply coupon extends checkout.



feature	Aggregation	Composition
Symbol	hollow diamond	filled diamond
ownership	part can exist independently of the whole.	part cannot exist without whole.
lifecycle	If whole deleted, the part survives	if whole is deleted, part also deleted
example	Library and book. if library closed book still exists	Human & head. if human dies, head also dies.

- 5b) Project Manager assure a client that the software product is high quality by:
- Process Transparency: Showing that the team follows a defined SQA plan.
 - Certifications: Adhering to standards like CMMI or ISO.
 - Deliverable: Providing regular test reports, bug tracking logs and prototype demonstrations.
 - SLA (Service Level Agreement): Guaranteeing specific performance and uptime metrics.

Elements of SQA:

- Standards: Defining the coding and document standards.
- Review and audit: peer reviews and technical audits to find defects.
- Testing: Systematic verification of software.
- change Management: Ensuring change's doesn't break existing quality.
- Education: Training the team on new tools and quality practices.

56) Measurement of Reliability and Availability
Reliability: Measures the probability of failure-free operation for a specific time.

Common metrics is Mean Time Between Failure (MTBF)

$$\text{Reliability} = \text{MTBF} / (1 + \text{MTBF}) \%$$

Availability: The percentage of time the system is operational.

$$\text{Availability} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} \times 100\%$$

MTTR (Mean Time to Repair)

$$\text{where, MTBF} = \frac{\text{total operational time}}{\text{no. of failures}}$$

$$\text{MTTR} = \frac{\text{Total maintenance downtime}}{\text{no. of repairs}}$$

Set of guidelines for Formal Technical Review (PTR):

- 1) Review the product, not producer
Avoid personal attacks on developer just review the quality of products.
- 2) Set an agenda and stick to it
Keep the meeting focused and brief.
- 3) Limit debate and rebuttal
Identify the problem, but don't try to solve it during the meeting
- 4) Take written notes
Use a review list to record every issue found
- 5) Keep the group small
involve only 3-5 people to maintain efficiency.

Auditing: An independent examination of software product or process it ensures it meets the standards or regulation.

Reporting: act of documenting and communicating the results of audits, tests on process status to stakeholders.

Reasons for SPI

- improve product quality and customer satisfaction
- reduce costs and development time
- enhances team productivity and morale
- ensures compliance with industry standards

Cost Estimation By Functional Point Analysis

Measurement Parameters	Examples
T = Number of External Input (EI) O = Number of External Output (EO) Q = Number of External Inquiries (EQ) F = Number of Internal Files (ILF) $N/2F$ = Number of External Interfaces (ALIF EIF)	input screen and tablets output screens and reports prompts & interrupts databases & directories shared databases and shared routines

5. Unadjusted Function Points

Function Units	Low	Avg	High
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

table value

कोई गलती नसे

List of General System Characteristics (GISC)

1. Data Communications
2. Distributed Data Processing
3. Performance
4. Heavily Used Configuration
5. Transaction Rate
6. On-line Data Entry
7. End-User Efficiency
8. On-line Update
9. Complex Processing
10. Reusability
11. Installation Ease
12. Operational "
13. Multiple Sites
14. Facilitate change

low 2
 $14 \times 0 = 0$
 $14 \times 3 = 42$ (avg)
 $14 \times 5 = 70$ (High)

 112

Cost Estimation By Functional Point Analysis

$$FP = UFP \times VAF$$

$$VAF = [0.65 + 0.01 \times \text{sum of GISC}]$$

↳ value Adjustment factor / CAF

$$UFP = 8 \times 9$$

$$4 \times 4$$

$$3 \times 2$$

$$7 \times 5$$

$$5 \times 3$$

$$81$$

$$GISC = 4 \times 0$$

$$4 \times 3$$

$$6 \times 5$$

$$42$$

$$VAF = 0.65 + 0.01 \times 42$$

$$= 1.07$$

$$FP = 81 \times 1.07 = 86.67$$

(Imp) Q. Given the following values compute function point when all complexity adjustment factor (CAF/VAF) and weighting factors are average (UFP)

User Input = 50

" output = 40

" inquiry = 35

Wiser files = 6

External interface = 4

productivity = 6.5 FPPM
 average salary = \$800
 per month

$$\text{Soln: } UFP = 50 \times 4 + 40 \times 5 + 35 \times 4 + 6 \times 10 + 4 \times 7$$

$$= 628$$

$$VAF = 0.65 + 0.01 \times (14 \times 3) = 1.07$$

$$FP = UFP \times VAF = 628 \times 1.07 = 671.96$$

$$\text{total effort} = \frac{FP}{\text{prod.}} = \frac{671.96}{6.5} = 103.38 \text{ PM}$$

$$\text{total cost} = 103.38 \times 800 = \$82704$$

Q2. Calculate the function point, productivity, with multiple processing factors 5, 1, 0, 4, 3, 5, 9, 3, 5, 2, 3, 9, 2 by using following given data:

Following given data:
 EI (Avg) = 22
 EO (Low) = 45
 EQ (High) = 06
 ILF (Avg) = 05
 ELF (Low) = 02

Effort = 37 PM

Software technical documents : 250 pages
 User related " : 120 "
 Budgeting / Cost : \$7520 per month

Soln: Here,

Given General System Characteristics =
 5, 1, 0, 4, 3, 5, 4, 3, 4, 5, 2, 3, 4, 2

$$\text{Now, VAF} = 0.65 + 0.01 \left[\frac{5 \times 1 + 1 \times 1 + 0 \times 1 + 4 \times 1 + 3 \times 1 + 5 \times 1 + 4 \times 1 + 3 \times 1 + 4 \times 1 + 5 \times 1 + 2 \times 1 + 3 \times 1 + 4 \times 1 + 2 \times 1}{2 + 2 + 4 + 5} \right]$$

$$= 0.65 + 0.01 \times 45$$

$$= 1.10$$

$$\text{UFP} = 22 \times 4 + 45 \times 4 + 6 \times 6 + 5 \times 10 + 2 \times 5$$

$$= 409$$

$$\therefore \text{FP} = \text{UFP} \times \text{VAF} = 409 \times 1.10 = 450 \text{ #}$$

$$\text{productivity} = \frac{40010}{37}$$

$$= 10.82$$

$$\text{cost per function point} = \frac{7520}{10.82} = \$695.01 \text{ per month per page}$$

Total pages = 370

$$\text{Documentation} = \frac{250}{370} \times \frac{370}{400.4} = 0.924$$

~~Given~~ EI=24, EO=46, EQ=8, F=4, EIF=2
 Effort = 36.9 PM, document = 256
 user document = 122, page cost = \$7744 per month
 Complexity factor: 4, 1, 0, 3, 3, 5, 4, 4, 3, 3, 2, 2, 4, 5

$$\text{VAF} = 0.65 + 0.01 \left[\frac{4 \times 1 + 1 \times 1 + 0 \times 1 + 3 + 3 + 5 + 4 + 4 + 3 + 3 + 2 + 2 + 4 + 5}{2 + 2 + 4 + 5} \right] = 1.08$$

$$\text{UFP} = 24 \times 4 + 46 \times 3 + 8 \times 4 + 4 \times 10 + 2 \times 5$$

$$= 552$$

$$\text{function point (FP)} = \text{VAF} \times \text{UFP} = 552 \times 1.08 = 596.16 \text{ #}$$

$$\text{productivity} = \frac{\text{FP}}{\text{E}} = \frac{596.16}{36.9} = 16.15 \text{ #/PM}$$

$$\text{documentation} = \frac{256 + 122}{\text{FP}} = 0.634 \text{ #/FP}$$

$$\text{cost per function} = \frac{7744}{16.15} = 479.508 \text{ #}$$

Q. Consider a software project with the following information domain characteristics for the calculation of function point metric.

Number of external input (I) = 30
 " " " output (O) = 60
 " " " external inquiries (E) = 23
 " " " files (F) = 08
 " " " external interfaces (N) = 2

It is given that the complexity weighting factors for I, O, E, F and N are 4, 5, 4, 10 and 7, respectively. It is also given that, out of fourteen value adjustment factors that influence the development effort, four factors are not applicable, each of other four factors has value 3 and each of the remaining factors has value 4. The computed value of the function point metrics.

Soln:

$$VAF = 0.65 + 0.01 \left(\frac{30 \times 4 + 60 \times 5 + 23 \times 4 + 8 \times 10 + 2 \times 7}{8 + 10 + 2 \times 7} \right) 4 \times 0 + 9 \times 0 + 9 \times 1$$

$$= 6.71$$

$$UFP = 4 \times 0 + 4 \times 3 + 6 \times 4 + 30 \times 4 + 60 \times 5 + 23 \times 4 + 8 \times 10$$

$$FP = \frac{36}{VAF \times UFP} = 6.71 \times 36 = 241.56$$

Cost Estimation By COCOMO Model

Types of COCOMO Model Project

Aspects	Organic	Semidetached	Embedded
	2-50	50-300	300 to above KLOC (Kilof codes)

1. Basic COCOMO Model
2. Intermediate
3. Detailed COCOMO Model

1. Basic COCOMO Model (कैली मॉडल)

$$E = a * (KLOC)^b * PM \quad (PM = \text{Person Month})$$

$$Tdev = \frac{E}{P} = \frac{a * (KLOC)^b * PM}{P}$$

$$\text{Person Required} = \frac{\text{Effort}}{\text{Time}}$$

Software Projects	a/a ₁	b/b ₂	c/b ₁	d/b ₂
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32
	3.16			

Q. Suppose that a basic project was estimated to be 8450 KLOC (Kilo Lines of Code).

Calculate effort and time for each of the 3 modes of development. All the constants value provided in the following table.

Solⁿ: KLOC = 950

1. for organic mode

$$\text{effort} = 2.4 \times (950)^{1.05} = 1465.83 = 1466 \text{ PM}$$

$$\text{dev Time} = 2.5 \times (1466)^{0.38} = 39.90 \text{ month}$$

$$\begin{aligned} \text{Average staffing (N)} &= E/T \\ &= \frac{1466}{39.90} = 36.74 \\ &= 36.65 \sim 37 \end{aligned}$$

2. for semi-detached

$$E = 3 \times (950)^{1.12} = 2810$$

$$T = 2.5 \times (2810)^{0.38} = 51$$

$$N = E/T = 55$$

3. for Embedded

$$E = 3.6 \times (950)^{1.20} = 5997$$

$$T = 2.5 \times (5997)^{0.32} = 39$$

$$N = 5997/39 = 154$$

(Preload case)

Q. A project size of 250 KLOC is to be developed. Software Development team has average experience on similar types of projects. The project schedule is not very tight. Calculate the effort, development time, average staff size and productivity of the project.

Solⁿ: The semi-detached mode is most appropriate mode; keeping in view the size, schedule and experience of development team.

$$\text{Hence, } E = 3 \times (250)^{1.12} = 1455$$

$$T = 2.5 \times (1455)^{0.35} = 32$$

$$SS = E/T = \frac{1455}{32} = 46 \text{ person}$$

productivity = $\frac{KLOC}{E} = \frac{250}{1455} = 0.172 \#$

2. Intermediate COCOMO Model

$$E = a * (KLOC)^b * EAF^c$$

$$T_{dev} = c * (E)^d$$

$$EAF = \text{Effort}$$

imp

- Test Case Development

CoTS

- Software Reuse

3. embedded

$$E = a * (KLOC)^b$$

$$T = c * (E)^d$$

EAF calculate from here

Cost Drivers	Ratings					
	Very low	Low	Normal	High	Very high	Extreme
<u>Product Attributes</u>						
Required software reliability	0.75	0.88	1	1.15	1.40	
Size of application database		0.94	1	1.08	1.16	
Complexity of the product	0.70	0.85	1	1.15	1.30	1.65
<u>Hardware Attributes</u>						
Runtime performance constraints			1	1.11	1.30	1.16
Memory constraints			1.0	1.06	1.21	1.56
Volatility of virtual machine environment		0.87	1	1.15	1.30	
Required turnabout time		0.87	1.0	1.07	1.15	
<u>Personal Attributes</u>						
Analyst capability	1.46	1.19	1.0	1.0	0.86	0.71
Applications experience	1.29	1.13	1.13	1.0	0.91	0.82
Software engineer capability	1.42	1.17	1.17	1.0	0.86	0.70
Virtual machine experience	1.21	1.10	1.10	1.0	0.90	
Programming language experience	1.14	1.07	1.07	1.0	0.95	
<u>Project Attributes</u>						
Application of software engineering	1.24	1.10	1.0	0.91	0.82	
Use of software tools	1.24	1.10	1.0	0.91	0.83	
Required development	1.23	1.08	1.0	1.04	1.10	

Q. A new project with estimated 400 KLOC to be developed. Project manager has a choice of hiring from two pools of developers. Very highly capable with very little experience in the programming language being used.

Or Developers of low quality but a lot of experience with the programming language. What is the impact of hiring all developers from one or the other pool?

Soln: KLOC = 400

Case 1: developer: high capable
programming language: little experience

$$EAF = 1.14 \times 0.92 = 1.0374 = 1$$

For organic model:

$$E = a \times (KLOC)^b \times EAF$$

$$= 2.4 \times (400) \times 1.0374$$

$$= 995.904 = 996$$

$$T_{dev} = 2.5 \times (995.904)^{0.38}$$

$$= 34.45$$

Case 1:

$$EAF = 0.82 \times 1.14 = 0.9348$$

$$E = 3712 \times 0.9348 = 3470$$

$$T_{dev} = 2.5 \times (3470)^{0.38} = 34$$

Case 2:

$$EAF = 1.29 \times 0.95 = 1.225$$

$$E = 2.8 \times (1.225) \times (400)$$

$$= 4528$$

$$D = 0.5(4528)^{0.32} = 36.9M$$

$$= 40M$$

MEC

project size = 9000 KLOC

productivity = 400 LOC/PM

labor rate = Rs 30,000 per month

$$\text{estimated effort} = \frac{LOC}{\text{productivity}} = \frac{90000}{400} = 225 \text{ PM}$$

$$\text{estimated project cost} = 225 \times 30,000$$

$$= 67,50,000 \text{ Rs.}$$

Consider a Software project with the 5-unadjusted function point having the value 20, 30, 40, 10, 2 respectively. The weights for these factors are of average type. It is also given that out of 14 GSC 7 factors has average value, 5 has highest value and remaining have negligible value. The effort for this analysis is 35pm and the cost is estimated as 10500\$. Find out the productivity and cost of each function.

Solⁿ:

given,

5-unadjusted function point = 20, 30, 40, 10, 2

$$\therefore \text{UFP} = 20 \times 4 + 30 \times 5 + 40 \times 4 + 10 \times 10 + 2 \times 7 \\ = 504$$

$$\text{sum of GCS} = 4 \times 3 + 5 \times 5 + 5 \times 0 \\ = 37$$

$$\therefore \text{VAF} = 0.65 + 0.01 \times 37 = 1.02 = 1$$

$$\therefore \text{FP} = \text{UFP} \times \text{VAF} = 504 \times 1.02 \\ = 514.08 = 514$$

$$\therefore \text{effort} = 35 \text{ pm} \\ \text{estimated cost} = 10500 \$$$

$$\text{productivity} = \frac{\text{FP}}{\text{effort}} = \frac{514.08}{35} = 14.68 = 15 \#$$

$$\text{cost of each function} = \frac{10500}{14.68} = 715.25 \$ \#$$

2. Suppose that the size of project 350KLOC. calculate effort, time, average staff and productivity for each of 3 modes of development.

Solⁿ: KLOC = 350

i) for organic mode:

$$\text{Effort (E)} = a * (\text{KLOC})^b = 2.4(350)^{1.05} = 1126 \text{ PM}$$

$$\text{Dev Time (T)} = c * (\frac{E}{\text{KLOC}})^d = 2.5(1126)^{0.38} = 36 \text{ M}$$

$$\text{Average Staff (N)} = E/T = \frac{1126}{36} = 31$$

$$\text{Productivity} = \text{KLOC}/E = 350/1126 = 0.310$$

ii) for semi-detached mode:

$$E = 3(350)^{2.12} = 2121 \text{ PM}$$

$$T = 2.5(2121)^{0.35} = 37 \text{ M}$$

$$N = \frac{2121}{37} = 57$$

$$\text{productivity} = \frac{350}{2121} = 0.165 \text{ \#}$$

iii) for embedded mode

$$E = 3.6(350)^{1.20} = 4066 \text{ PM}$$

$$T = 2.5(4066)^{0.32} = 36 \text{ M}$$

$$N = \frac{4066}{36} = 113$$

$$\text{productivity} = \frac{350}{4066} = 0.086 \text{ \#}$$

3. A new project with estimated 450 KLOC hybrid system has to be developed. Project manager has a task to deliver very high the reliable software with a good efficient performance also if that type of system is not possible then the reliability of software can be of low quality but the runtime performance should be very high. For the first type of system the developers has to be of nominal experience while using applications but he should be good at using software tools. Similarly if 2nd type of system has to be developed, the developer should have a ~~project~~ great experience of application handling while he can be average in using software tools.

Soln: project size (KLOC) = 450
// embedded

Case 1:

application experience: nominal
use of software tools: good

$$EAF = 1.13 \times 0.91 = 1.02$$

$$E = 3.6 (450)^{1.20} \times EAF = 5497 PM \times 1.02 = 5606.94 = 5607 PM$$

$$T = 2.5 (5607)^{0.32} = 39.58 M = 40 M \#$$

Case 2: $0.91 \times 0.91 = 0.828$

$$EAF = 3.6 (450)^{1.20} \times 0.828 = 4552$$

$$E = 4552 PM$$

$$T = 2.5 (4552)^{0.32} = 37 M \#$$

Check
= 11x12

2024, fall

Date:

Page:

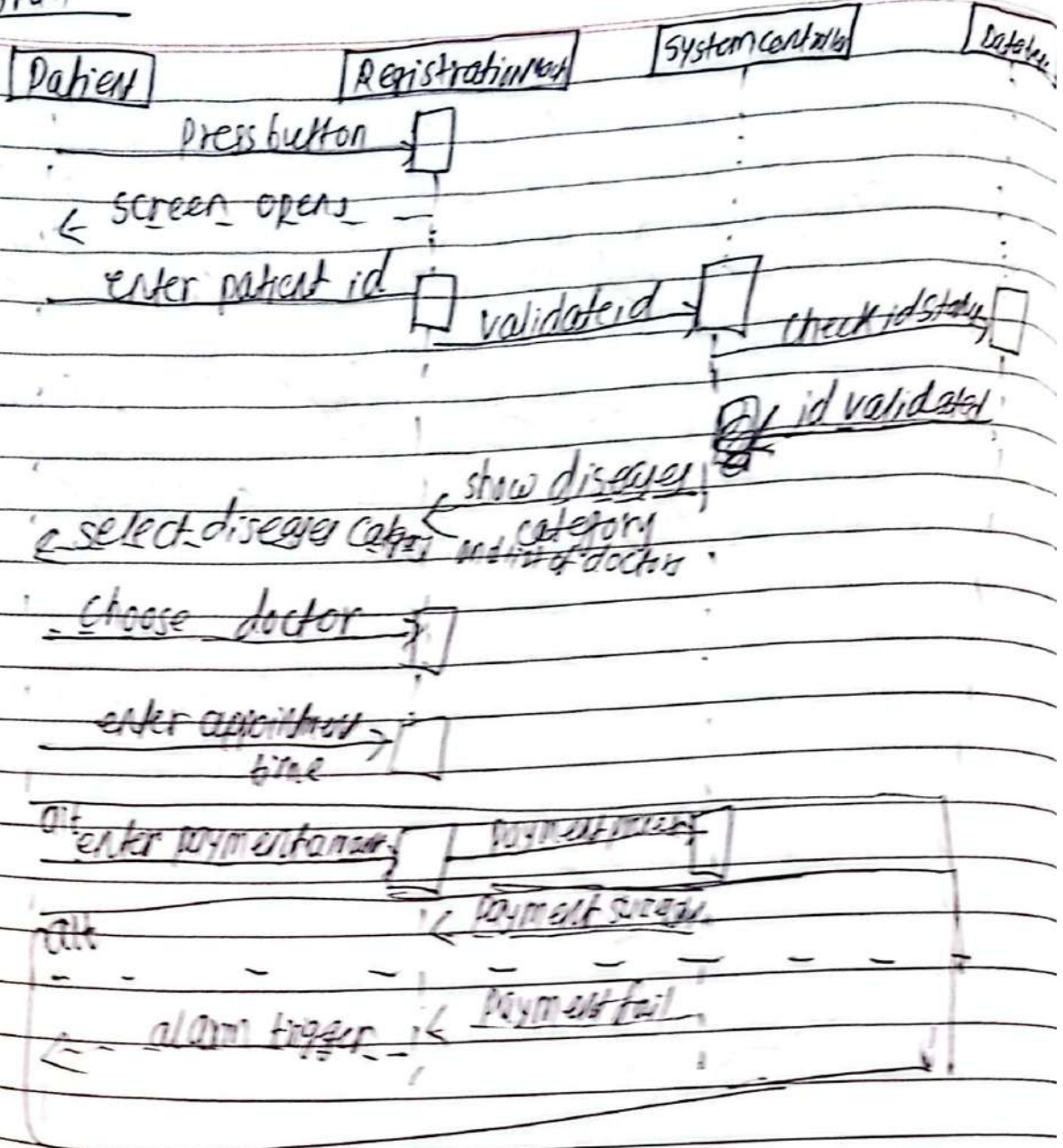


fig: Sequence diagram

3.

So,
pers
is
can be
perform

2025 Spring

Date:

Page:

Sequence diagram for ticket booking process.

A user searches for available flights by entering travel details. The system contacts the airline database and shows available flights. The user selects a flight and provides passenger details. The system requests payment from the payment gateway. Upon successful payment, the ticket is issued, and a confirmation email is sent.

S/N:

